

Deep Learning & Generative AI Project

Messy Mashup – Music Genre Classification

1. Abstract / Executive Summary

This project addresses the Messy Mashup genre classification problem, where mashup audio clips composed of multiple stems must be classified into one of ten genres.

Several approaches were explored including rule-based classification, classical machine learning, CNNs, CRNNs, and a fine-tuned model (AST).

The best performing model was a fine-tuned Audio Spectrogram Transformer, achieving a Macro F1-score of 0.87619, significantly outperforming custom neural networks and classical models.

2. Introduction

Problem Statement

The task is to classify mashup audio tracks into 10 music genres. Each track is created by mixing stems (drums, vocals, bass, other) and may include environmental noise.

Project Objective

The goals of this project were:

- Analyze the dataset through EDA
- Develop baseline, deep learning models
- Apply transformer-based transfer learning
- Compare model performance

Report Structure

The report presents the dataset analysis, feature extraction strategy, model architectures, performance comparison, and conclusions.

3. Dataset & Preprocessing

Dataset Description

The dataset contains 1000 songs across 10 genres, with each song consisting of four stems. Each audio track is approximately 30 seconds long. The ESC-50 dataset (2000 environmental sounds across 50 classes) was used for data augmentation.

Exploratory Data Analysis (EDA)

- The dataset is perfectly balanced across genres.
- Average audio length is 30 seconds for all stems.
- Some stems contain long silent segments, especially vocals in classical and jazz genres.
- Mel spectrograms reveal distinctive frequency patterns across genres.

Data Preprocessing

- Silence trimming
- Amplitude normalization
- Fixed-length padding/truncation
- Conversion to mel spectrogram features

Data Augmentation

Two augmentation methods were used:

1. Random stem mixing to generate new mashups.
2. Noise injection using ESC-50 audio with random SNR levels.

This increased dataset diversity and improved generalization.

4. Tokenization Strategy

For the transformer model, audio was converted to spectrogram patch tokens using the AST Feature Extractor from Hugging Face.

The extractor processes raw audio by:

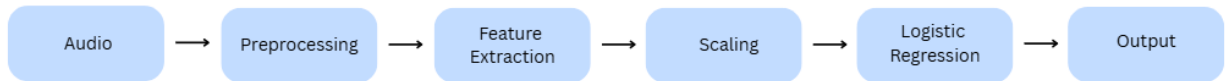
- Resampling to 16 kHz
- Converting to log-mel spectrograms
- Splitting into patches used as transformer tokens

Feature extraction was precomputed and stored in chunks to avoid memory issues.

5. Modeling & Experimentation

5.1 Model 1 – Classical ML Model (Logistic Regression)

Architecture:

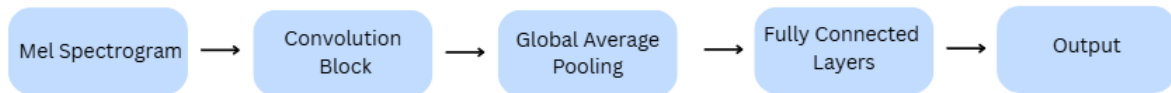


Salient Points:

- Uses manually extracted audio features
 - Fast and simple to train
 - Good baseline but cannot learn complex patterns from raw audio
-

5.2 Model 2 – Custom Deep Learning Model (CNN)

Architecture:

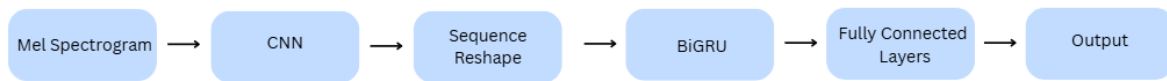


Salient Points:

- Learns patterns directly from spectrogram images
 - Good at capturing local sound features
 - Does not handle long-term timing patterns well
-

5.3 Model 3 – RNN-Based Model (CRNN)

Architecture:



Salient Points:

- Combines CNN and RNN
 - CNN learns features, GRU learns timing patterns
 - Better than CNN for audio sequences
-

5.4 Model 4 – Fine-Tuned Transformer Model (AST)

Architecture:



Salient Points:

- Uses attention to understand full audio context
- Works well for long audio clips
- Uses pretrained knowledge for better results

Fine-Tuning Strategy:

- Lower layers frozen
 - Encoder layers 6–11 unfrozen
 - Custom classifier head added
 - Optimizer: AdamW
 - Learning rate: 3e-5
-

6. Performance & Comparative Analysis

Evaluation Metric

The primary metric used was Macro F1-score, ensuring equal importance for all genres.

Training Performance



Wandb runs on CNN, CRNN and AST models recorded with F1, Accuracy and loss for training and validation

Model Comparison

Model	Training F1	Validation F1
Logistic Regression	0.6889	0.3561
CNN	0.63302	0.6942
CRNN	0.87829	0.8142
AST	0.99273	0.9593

Kaggle Performance

Final Leaderboard Score	0.87619
Rank	184

Observations

- Classical ML models struggle with complex audio patterns.
 - CNN models significantly improve performance.
 - CRNN models capture temporal dependencies.
 - Transformer models achieve the best results due to pretraining and attention mechanisms.
-

7. Conclusion & Future Work

Key Learnings

Data augmentation improved model robustness to noisy mashups. Incorporating temporal modeling (CRNN) significantly improved performance over CNNs. Transformer-based models achieved the best results due to pretrained representations and global attention.

Challenges Faced

Training transformer models required high computational resources and large memory for audio features. This was mitigated using chunked feature extraction and selective layer fine-tuning.

Future Work

Performance could be improved through ensemble models, stronger spectrogram augmentation methods, hyperparameter tuning, and experimenting with larger pretrained audio transformers.

8. References

- Hugging Face Transformers
 - Librosa Audio Processing Library
 - ESC-50 Dataset
 - PyTorch Deep Learning Framework
-